SQL / PostgreSQL

Task	Command	Notes
Install	`brew install postgresql@16`	`brew services start postgresql@16`
CLI client	`psql -d dbname`	`-U user -h host -p port`
Create database	`createdb name`	or `CREATE DATABASE name;`
Drop database	`dropdb name`	or `DROP DATABASE name;`
Dump	`pg_dump db > f.sql`	`-Fc` for custom format
Restore	`pg_restore -d db f.dump`	or `psql -f f.sql`
GUI	`pgcli` / DBeaver	`brew install pgcli`

psql Meta-Commands

`\l`	List databases
`\dt`	List tables
`\d tbl`	Describe table
`\di`	List indexes
`\dv`	List views
`\df`	List functions
`\dn`	List schemas
`\du`	List roles
`\dx`	List extensions
`\x`	Toggle expanded output
`\timing`	Toggle query timing
`\e`	Edit in $EDITOR
`\i f.sql`	Execute file
`\copy`	Client-side COPY
`\q`	Quit

SELECT Basics

`SELECT * FROM t`	All columns
`SELECT a, b FROM t`	Specific columns
`WHERE cond`	Filter rows
`ORDER BY col`	Sort
`ASC|DESC`	
`LIMIT n OFFSET m`	Paginate
`DISTINCT`	Unique rows
`AS alias`	Column/table alias

Filtering

`= <> < > <= >=`	Comparison
`AND / OR / NOT`	Logical
`IN (a, b, c)`	Set membership
`BETWEEN a AND b`	Range (inclusive)
`LIKE '%pat%'`	Pattern (% = any, _ = one)
`ILIKE`	Case-insensitive LIKE
`IS NULL / IS NOT NULL`	Null check
`EXISTS (subquery)`	Existence test
`ANY / ALL (subquery)`	Quantified compare

Joins

`INNER JOIN t ON cond`	Matching rows both
`LEFT JOIN t ON cond`	All left + matching right
`RIGHT JOIN t ON cond`	All right + matching left
`FULL JOIN t ON cond`	All rows both sides
`CROSS JOIN t`	Cartesian product
`LATERAL (subquery)`	Correlated subquery join
`NATURAL JOIN t`	Join on same-named cols
`USING (col)`	Join on named column

Aggregation

`COUNT(*) / COUNT(col)`	Row / non-null count
`SUM(col)`	Total
`AVG(col)`	Average
`MIN(col) / MAX(col)`	Min / max
`GROUP BY col`	Group rows
`HAVING cond`	Filter groups
`array_agg(col)`	Collect into array
`string_agg(col, ',')`	Concatenate strings
`jsonb_agg(col)`	Collect into JSON array

Window Functions

`OVER (PARTITION BY ...)`	Window definition
`OVER (ORDER BY ...)`	Ordered window
`ROW_NUMBER()`	Sequential number
`RANK() / DENSE_RANK()`	Rank with/without gaps
`LAG(col, n) / LEAD(col, n)`	Previous / next row
`FIRST_VALUE / LAST_VALUE`	First / last in window
`SUM(col) OVER (...)`	Running total
`NTILE(n)`	Divide into n buckets

Subqueries & CTEs

`WITH cte AS (...)`	Common table expression
`WITH RECURSIVE ...`	Recursive CTE
`(SELECT ...) AS sub`	Subquery in FROM
`WHERE x IN (SELECT ...)`	Subquery in WHERE

Set Operations

`UNION`	Combine (deduplicated)
`UNION ALL`	Combine (keep dupes)
`INTERSECT`	Common rows
`EXCEPT`	Rows in first not second

DML — Data Modification

`INSERT INTO t (cols) VALUES`	Insert row
`INSERT ... RETURNING *`	Return inserted
`UPDATE t SET c=v WHERE`	Update rows
`DELETE FROM t WHERE`	Delete rows
`UPSERT: ON CONFLICT ...`	Insert or update
`DO UPDATE SET c=EXCLUDED.c`	Upsert action
`DO NOTHING`	Skip conflicts
`COPY t FROM 'f.csv' CSV`	Bulk load
`TRUNCATE t`	Fast delete all

DDL — Schema

`CREATE TABLE t (...)`	Create table
`ALTER TABLE t ADD col type`	Add column
`ALTER TABLE t DROP col`	Drop column
`ALTER TABLE t RENAME`	Rename table/column
`DROP TABLE t`	Drop table
`CREATE INDEX ON t (col)`	B-tree index
`CREATE UNIQUE INDEX`	Unique constraint
`CREATE VIEW v AS SELECT`	Create view
`CREATE MATERIALIZED VIEW`	Cached query
`REFRESH MATERIALIZED VIEW`	Rebuild cached

Constraints

`PRIMARY KEY`	Unique + not null
`UNIQUE`	No duplicates
`NOT NULL`	Required
`CHECK (cond)`	Custom constraint
`REFERENCES t(col)`	Foreign key
`DEFAULT val`	Default value
`GENERATED ALWAYS AS (expr)`	Computed column

PostgreSQL — Types, Functions & PostGIS

PostgreSQL Types

`integer / bigint`	32/64-bit int
`numeric(p,s)`	Exact decimal
`real / double precision`	32/64-bit float
`text / varchar(n)`	String
`boolean`	true / false
`date / time / timestamp`	Date/time
`timestamptz`	Timestamp + timezone
`interval`	Time duration
`uuid`	UUID
`jsonb`	Binary JSON
`int[] / text[]`	Array types
`bytea`	Binary data
`serial / bigserial`	Auto-increment
`tsquery / tsvector`	Full-text search

JSONB Operations

`j->'key'`	Text value by key
`j->'key'`	JSON value by key
`j->0`	Array element (text)
`j #> '{a,b}'`	Nested path (JSON)
`j #>> '{a,b}'`	Nested path (text)
`j @> '{"k":"v"}'`	Contains
`j ? 'key'`	Key exists
`jsonb_set(j, path, val)`	Update value
`j || '{"k":"v"}'`	Merge
`jsonb_each(j)`	Key-value rows
`jsonb_array_elements(j)`	Array to rows
`jsonb_build_object(k,v)`	Build object
`to_jsonb(record)`	Row to JSON

Array Operations

`a[1]`	Index (1-based)
`a[1:3]`	Slice
`array_length(a, 1)`	Length
`a || b`	Concatenate
`ANY(a)`	Any element matches
`ALL(a)`	All elements match
`unnest(a)`	Array to rows
`array_agg(col)`	Rows to array
`a @> b`	a contains b

String Functions

`length(s)`	String length
`upper(s) / lower(s)`	Case convert
`trim(s)`	Trim whitespace
`substring(s, i, n)`	Extract substring
`replace(s, from, to)`	Replace
`split_part(s, d, n)`	Split and take nth
`regexp_matches(s, pat)`	Regex extract
`regexp_replace(s, p, r)`	Regex replace
`concat_ws(',', a, b)`	Join with separator
`format('%s is %s', a, b)`	Printf-style

Date / Time Functions

`now() / current_timestamp`	Current time
`current_date`	Today
`extract(year FROM ts)`	Extract field
`date_trunc('month', ts)`	Truncate to month
`ts + interval '1 day'`	Add interval
`age(ts1, ts2)`	Difference
`to_char(ts, 'YYYY-MM-DD')`	Format
`generate_series(a, b, step)`	Date range

Indexes

`B-tree`	Default (equality, range)
`GIN`	jsonb, arrays, full-text
`GiST`	Geometry, range, full-text
`BRIN`	Large sorted tables
`CREATE INDEX CONCURRENTLY`	No table lock
`WHERE cond`	Partial index
`(expr)`	Expression index
`INCLUDE (col)`	Covering index

Performance

`EXPLAIN ANALYZE q`	Query plan + timing
`EXPLAIN (BUFFERS) q`	I/O stats
`pg_stat_statements`	Query stats extension
`VACUUM ANALYZE t`	Update stats
`pg_stat_user_tables`	Table stats

Transactions

`BEGIN`	Start transaction
`COMMIT`	Save changes
`ROLLBACK`	Undo changes
`SAVEPOINT s`	Named savepoint
`ROLLBACK TO s`	Undo to savepoint
`SET TRANSACTION ISOLATION`	Read committed (default)
`SERIALIZABLE`	Strictest isolation

PostGIS — Setup

`CREATE EXTENSION postgis`	Enable PostGIS
`geometry / geography`	Spatial column types
`SRID 4326`	WGS 84 (lon/lat)
`SRID 3857`	Web Mercator

PostGIS — Constructors

`ST_Point(x, y)`	Point geometry
`ST_MakeLine(a, b)`	Line from points
`ST_MakePolygon(ring)`	Polygon from ring
`ST_GeomFromText(wkt, srid)`	From WKT
`ST_GeomFromGeoJSON(j)`	From GeoJSON
`ST_SetSRID(g, srid)`	Set coordinate ref

PostGIS — Measurements

`ST_Distance(a, b)`	Distance
`ST_Area(g)`	Area
`ST_Length(g)`	Line length
`ST_Perimeter(g)`	Perimeter
`ST_Centroid(g)`	Centroid
`ST_Azimuth(a, b)`	Bearing angle

PostGIS — Spatial Predicates

`ST_Contains(a, b)`	a contains b
`ST_Within(a, b)`	a within b
`ST_Intersects(a, b)`	Any overlap
`ST_DWithin(a, b, d)`	Within distance d
`ST_Crosses(a, b)`	Crosses
`ST_Touches(a, b)`	Touches boundary
`ST_Disjoint(a, b)`	No intersection
`ST_Covers(a, b)`	Covers

PostGIS — Operations

`ST_Buffer(g, d)`	Buffer geometry
`ST_Intersection(a, b)`	Intersection
`ST_Union(a, b)`	Union
`ST_Difference(a, b)`	Difference
`ST_Transform(g, srid)`	Reproject
`ST_Simplify(g, tol)`	Simplify geometry
`ST_Envelope(g)`	Bounding box
`ST_ConvexHull(g)`	Convex hull
`ST_Snap(a, b, tol)`	Snap to geometry

PostGIS — Output

`ST_AsText(g)`	WKT
`ST_AsGeoJSON(g)`	GeoJSON
`ST_AsMVT(row)`	Mapbox vector tile
`ST_X(pt) / ST_Y(pt)`	Extract coordinates
`ST_SRID(g)`	Get SRID
`ST_GeometryType(g)`	Type name

PostGIS — Spatial Indexes

`CREATE INDEX ON t USING GiST (geom)`	Spatial index
`a && b`	Bounding box overlap
`ORDER BY g <-> ref`	KNN distance order